REA-RL: Reflection-Aware Online Reinforcement Learning for Efficient Large Reasoning Models

Hexuan Deng[1] **Wenxiang Jiao** **Xuebo Liu**[1*] **Jun Rao**[1] **Min Zhang**[1]

[1]Institute of Computing and Intelligence, Harbin Institute of Technology, Shenzhen, China
{hxuandeng,wenxiangjiaonju,rao7jun}@gmail.com,
{liuxuebo,zhangmin2021}@hit.edu.cn

Abstract

Large Reasoning Models (LRMs) demonstrate strong performance in complex tasks but often face the challenge of *overthinking*, leading to substantially high inference costs. Existing approaches synthesize shorter reasoning responses for LRMs to learn, but are inefficient for online usage due to the time-consuming data generation and filtering processes. Meanwhile, online reinforcement learning mainly adopts a length reward to encourage short reasoning responses, but tends to lose the reflection ability and harm the performance. To address these issues, we propose REA-RL, which introduces a small reflection model for efficient scaling in online training, offering both parallel sampling and sequential revision. Besides, a reflection reward is designed to further prevent LRMs from favoring short yet non-reflective responses. Experiments show that both methods maintain or enhance performance while significantly improving inference efficiency. Their combination achieves a good balance between performance and efficiency, reducing inference costs by 35% without compromising performance. Further analysis demonstrates that our methods are effective by maintaining reflection frequency for hard problems while appropriately reducing it for simpler ones without losing reflection ability. Codes are available at https://github.com/hexuandeng/REA-RL.

1 Introduction

Large Reasoning Models (LRMs) have demonstrated impressive performance in downstream applications [16, 18, 38]. Their human-like deliberation and insightful self-reflection ability facilitate thorough question consideration and verification, thereby improving performance on complex reasoning tasks [8, 10]. However, this often leads to excessive reasoning with minimal performance benefit, i.e., *overthinking*, substantially increasing the inference cost [5, 31].

Prior research [11, 28, 39] attempts to generate shorter reasoning responses for supervised fine-tuning (SFT) or reinforcement learning (RL) to encourage concise response generation. However, this method relies on static datasets, the distribution of which may deviate from the trained model, especially later in training, leading to suboptimal results [37]. Furthermore, the time-consuming data generation and filtering processes severely limit its feasibility as an online data generation solution.

Towards these problems, another line of work [2, 24, 35] employs online reinforcement learning. To enhance the efficiency of online data generation, multiple reasoning paths are sampled in parallel for a single query, with accuracy and length rewards used to encourage correct and concise responses [43, 46]. However, this self-iterative training paradigm can lead to unpredictable behavior. As

* Corresponding Author

Preprint. Under review.

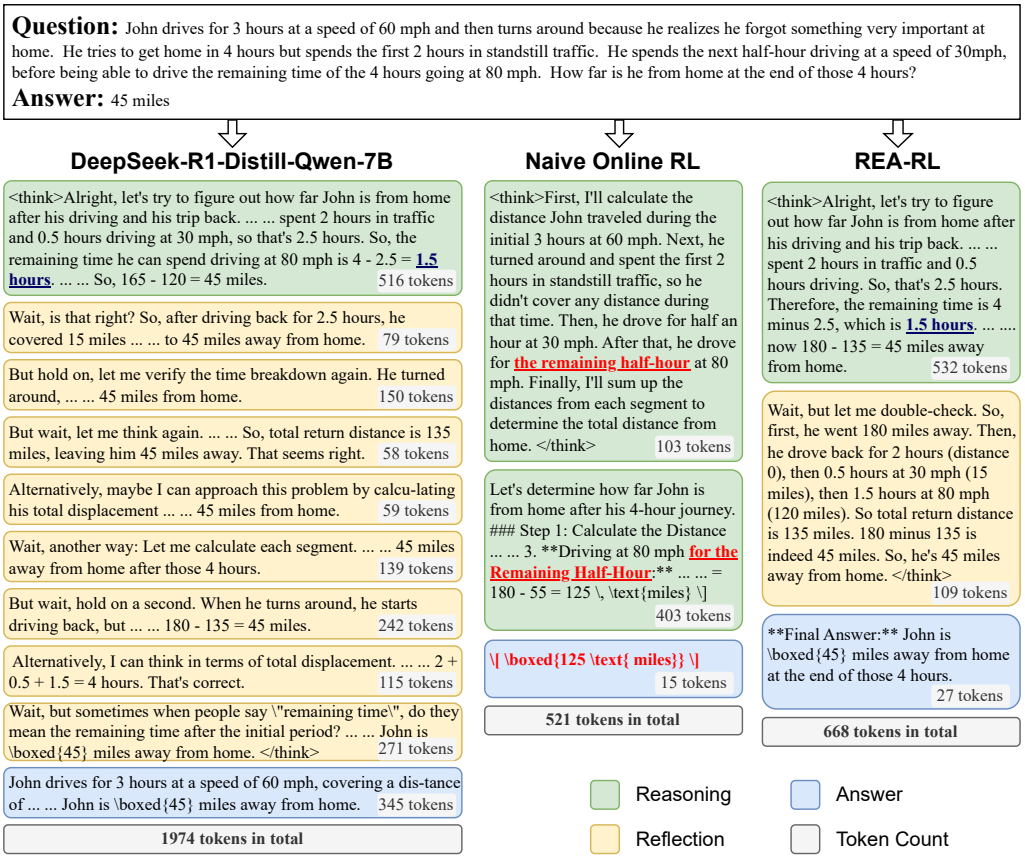

Figure 1: Overthinking and non-reflective cases from GSM8k. The left column shows the output of DeepSeek-R1-Distill-Qwen-7B, which reflects eight times before finishing generation. The middle column presents the output after online RL training using length rewards, which only spends 103 tokens in "think" part and no reflection, where an error occurs (underlined). The right column shows our output, which uses a similar budget to R1-7B in reasoning but only performs a single reflection.

illustrated in Figure 1, it can cause the LRM to completely lose its reflection capability, reverting to a naive chain-of-thought style, resulting in suboptimal performance on complex reasoning tasks.

In this paper, we propose `REA-RL`, a reflection-aware reinforcement learning approach that integrates the strengths of aforementioned ones. First, we introduce a small **reflection model** into the online RL process to identify the first reflection position in a sampled response that is very likely to derive the final answer, and truncate the response to a shorter one for optimization. It enables us to perform both parallel sampling and sequential revisions, which has been demonstrated to achieve computationally optimal test-time scaling during inference [36]. Second, we introduce a **reflection reward** to prevent the models from favoring short yet non-reflective responses. The reward is calculated based on the density of keywords (e.g., "wait", "alternatively") that signal reflective thinking in the response.

Experimental results reveal that employing only the length reward in online RL leads to a substantial performance degradation. In contrast, both the reflection model and the reflection reward significantly improve performance and prevent non-reflective behavior. The reflection reward is better at maintaining performance, while the reflection model achieves higher efficiency. Combining both approaches achieves a 35% response shortening on all datasets without performance reduction, and 44% on easier datasets. Further analysis demonstrates that these improvements stem from maintaining the reflection tendency on difficult questions and appropriately reducing reflection on simple questions without losing reflection ability. Our contributions are summarized as follows:

- We design an efficient overthinking detection method, enabling small models to perform this task. Furthermore, we train a **reflection model** for the online generation of shorter response revisions, facilitating both parallel sampling and sequential revision to achieve more efficient scaling.

- We design a **reflection reward** to prevent non-reflective behavior in online RL and significantly enhance model performance compared to using length reward only.
- Results demonstrate that each method reduces inference cost while preserving model performance. Combining both methods achieves a 35% response shortening without performance degradation, reducing the reflection frequency for simple questions while maintaining it for difficult ones.

2 Related Work

Large reasoning models. System 1 large language models (LLMs) have shown remarkable capabilities across diverse tasks [1, 21, 42]. However, complex reasoning tasks like advanced mathematics and formal logic necessitate more structured analysis [9, 40]. To address this, LRMs have emerged [8, 10, 16]. These models explicitly generate intermediate reasoning steps, i.e., test-time scaling [4, 27], before providing a final answer, incorporating self-reflection and error correction to enhance their performance on complex reasoning tasks [20, 26]. However, this enhanced capability comes at the cost of slower and more verbose reasoning processes [31, 41].

Efficient reasoning with response revision. Training on revised responses with less overthinking via SFT and RL enhances reasoning efficiency. Several methods employ parallel sampling. Munkhbat et al. [28] construct concise reasoning via best-of-N sampling and few-shot conditioning. Yu et al. [44] skip reasoning for high-confidence samples when facing perturbation. Other methods utilize stronger models to generate revisions. C3ot [17] employs GPT-4 [1] to compress reasoning while preserving key information. Chen et al. [5] use strong LLMs to detect overthinking and generate preference data. Han et al. [11] search for the optimal token budget and generate corresponding length responses using multiple strong LLMs. Still others rely on heuristic algorithms and time-consuming post-verification and filtering. SPIRIT-FT [7] identifies crucial steps using perplexity difference after removing the step. LM-skip [23] stimulates step-skipping behavior by iteratively refining models to generate shorter and accurate reasoning paths. TokenSkip [39] achieves fine-grained compression by omitting less important tokens in CoT outputs detected by LLMLingua-2 [29]. However, these methods often suffer from low efficiency due to complex generation and filtering processes, leading to more than double the generation cost compared to naive sampling, rendering their efficiency too low for online settings.

Efficient reasoning with length reward. DeepSeek R1 [10] achieves promising results using only accuracy and format rewards with online RL, demonstrating its effectiveness. Several methods also use RL to enhance the efficiency of LLMs with length rewards, often normalized by a baseline budget. O1-Pruner [24] estimates the budget from a reference model, while Arora and Zanette [3] do so from per-prompt groups. Kimi K1.5 [8] normalizes length with the minimum and maximum lengths of generations, while GRPO-LEAD [46] bases it on the length distribution. Other approaches predict the required maximum length budget. DAST [35] estimates the budget based on problem difficulty. Yeo et al. [43] propose a cosine reward that incentivizes meaningful reasoning while only penalizing excessive length. L1 [2] expects the model to follow the maximum length limit in the prompt and designs a target-aware reward. Finally, Liu et al. [22] collect preference pairs and construct rewards based on relative length. However, most methods depend solely on multiple samplings of a query, which may cause unpredictable behavior in RL. To address this, we provide online revision and refined reward design, better guiding the model's optimization direction.

3 Overthinking Detection

Previous works [5, 8, 31] observe that LRMs like QwQ-32B-Preview [38] and DeepSeek-R1 [10] tend to generate more solution rounds for simple math problems, allocating excessive computational resources with limited utility. We extend this analysis to smaller LRMs, and propose an effective detection method that does not necessitate the use of powerful closed-source LLMs for detection.

3.1 Auto-Detection of Overthinking

Problem definition. As illustrated in Figure 1, we observe a similar phenomenon in Deepseek-R1-Distill-Qwen-7B (R1-7B): the model tends to engage in excessive reflection after completing

Method	GSM8K		Math500		Gaokao23		Amc23		Olympiad		Aime24		Average	
	Acc↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓
Original	**91.66**	100_{1344}	92.00	100_{3893}	**81.82**	100_{3785}	88.12	100_{5840}	60.15	100_{7549}	48.33	100_{10460}	77.01	100_{5478}
Fixed Trunc (90)	89.39	80.51	91.40	83.51	81.30	83.20	85.00	85.68	59.41	86.77	44.17	88.72	75.11	84.73
Fixed Trunc (80)	88.25	72.47	89.60	75.52	79.22	75.22	78.44	77.67	58.67	77.92	37.08	80.09	71.88	76.48
Fixed Trunc (70)	87.26	64.73	86.00	66.76	75.58	66.61	73.12	68.99	53.19	**69.10**	33.33	**71.02**	68.08	67.87
Model Revise	89.46	62.43	**93.20**	71.85	80.52	70.41	**89.06**	79.95	**60.74**	83.06	50.83	93.59	**77.30**	76.88
+ Gold	88.78	**54.09**	92.00	**57.80**	79.48	**59.71**	87.19	**67.71**	60.15	73.48	**51.67**	92.31	76.54	**67.52**

Table 1: Performance and efficiency after model revision. "Original" denotes the R1-7B performance before revision. "Acc" represents the response accuracy, and "TR" represents the efficiency compared to the original ones, calculated by the token ratio compared to the "Original". Therefore, the TR in the first row is 100, representing no revision, and its subscript indicates the corresponding average token length. "Average" is the macro-average across datasets. The best and second-best results are **bold** and underline, respectively. Abbreviations of methods are defined in §3.2.

reasoning and obtaining the correct answer, leading to overthinking. In preliminary experiments, we find that weaker LLMs struggle to extract the boundaries of each reflection. However, we note that the conclusion of both reasoning and reflection is often a restatement of the answer. Therefore, we prompt LLMs to determine if each part of the response contains the correct answer, which is easier for LLMs. The thought process preceding the first occurrence of the correct answer is considered effective reasoning, while each subsequent appearance is an additional reflection, i.e., overthinking.

Detection method. We employ Qwen2.5-32B-Instruct [33] to identify these positions within the "think" part of LRMs. Specifically, we segment the think part into smaller chunks and provide the input question along with all these chunks as input, prompting the model to determine whether each chunk contains the correct answer. Subsequently, to ensure accuracy, we apply a further filtering step: chunks containing the answer are verified again one by one with the LLM, and only those confirmed as positive in both detections are considered correct. We design prompts with and without the inclusion of the gold answer as input, detailed in Appendix A.

Response revision. The tokens after the first correct answer are identified as overthinking and subsequently removed. Following this, we generate the revision by forcibly terminating the "think" part and compelling the LRM to continue generating the final answer, prefixed with "**Final Answer:**", which yields a revision without overthinking. For verification, the revision is deemed correct if the LRM can accurately generate the final answer. Finally, to prevent excessively long generations from skewing the average length, we limit the generation budget to 16k tokens.

3.2 Efficiency of Auto-Detection

Experimental setup. "Model Revise" and "+Gold" represent revision using our method without and with the gold answer as input, respectively. Our baseline for comparison is "Fixed Trunc (G)", which denotes fixed truncation of the original generated response, retaining $n\%$ of the thinking tokens and truncating the response at the nearest newline. Subsequently, similar to response revision, the "think" part is terminated, and the LRM is compelled to generate the final answer.

Evaluation dataset. We follow Yang et al. [42] to evaluate our approach on six math test sets, ordered by difficulty: **GSM8K** [6], 8.5K grade school math problems; **MATH500** [13], 500 challenging high school competition problems; **Gaokao23** [19], English-translated math questions from the 2023 Chinese Gaokao exam; **Amc23**[2], 2023 American Mathematics Competitions; **Olympiad** [12], Olympiad-level math and science questions; and **Aime24**[3], 2024 American Invitational Mathematics Examination. We employ the math validator provided by rStar [9]. Considering the limited size of Amc23 and Aime24, we sample 8 paths for each question to mitigate randomness.

Awesome auto-detection ability. Results before and after revision are in Table 1. Without the gold answer as input, we can automatically remove 23% of tokens without performance degradation.

[2] https://huggingface.co/datasets/AI-MO/aimo-validation-amc

[3] https://huggingface.co/datasets/AI-MO/aimo-validation-aime

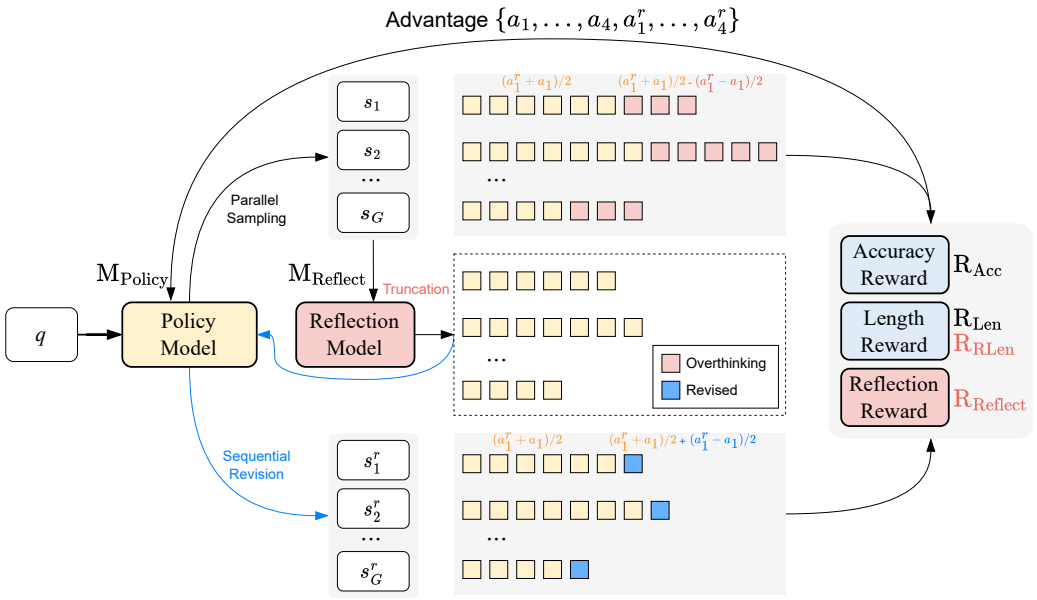

Figure 2: Workflow of REA-RL. The upper section illustrates the baseline GRPO process, which *parallel sampling* G completions $\{s_1, ..., s_G\}$, and optimize using an accuracy reward and a length reward (R_{Len}). The bottom section shows our reflection model ($M_{Reflect}$) performing *sequential revision*. From the G generated completions, the reflection model identifies and truncates overthinking tokens (red segment), keeps only the preceding segments (yellow segment), and then lets the policy model (M_{Policy}) complete the answer generation (blue segment), resulting in the revisions $\{s_1^r, ..., s_G^r\}$, with cases shown in Appendix B.1. We also introduce a reflection reward ($R_{Reflect}$) and a refined length reward (R_{RLen}). Finally, the advantages $\{a_1, ..., a_G, a_1^r, ..., a_G^r\}$ are calculated based on both the initial and revised completions, and used to train the policy model.

When provided, 32% of tokens are removed with a minor performance decrease. However, fixed truncation shows a considerable performance decline with increasing truncation ratios. These results demonstrate the effectiveness of our detection method and underscore the severity of the overthinking problem.

Greater overthinking on simpler problems. Across the two easiest, two medium, and two hardest math datasets, the overthinking tokens our method detects decrease progressively, at 38%, 31%, and 14%, respectively. This indicates that the overthinking problem is more prevalent in simpler datasets and less prevalent in more challenging ones.

4 Reflection-Aware Online Reinforcement Learning

To maintain model performance while reducing inference cost, we employ **online RL** to enhance the distributional alignment between the data and the model (§4.1). Furthermore, to enable both parallel sampling and sequential revision for online RL data generation, thereby facilitating better scaling and guiding the direction of online RL optimization, we introduce an efficient **reflection model** that provides revisions online (§4.2). Finally, we refine the reward to include a **reflection reward** to penalize non-reflective behavior (§4.3). The workflow is illustrated in Figure 2.

4.1 Online Reinforcement Learning

GRPO training. We adopt Grouped Relative Policy Optimization (GRPO, 34). Specifically, for each question in a given dataset, GRPO samples a group of outputs in parallel from the old policy model and then optimizes the current policy model by increasing the probability of outputs with high advantage and decreasing those with low advantage, which is normalized based on the reward.

Accuracy reward. Consistent with DeepSeek R1, we employ the rule-based accuracy reward to mitigate reward hacking. Specifically, we extract the final answer and compare it with the gold answer to verify its correctness. The reward is 1 for a correct answer and 0 otherwise.

Length reward. Following Kimi K1.5 [8], we incorporate a length reward to improve the LRM's efficiency. Formally, given a group of sampled responses $\{s_1, ..., s_G\}$, where max_len is the length of the longest response and min_len is that of the shortest, the length reward for the i-th response is:

$$R_{\text{Len}}(r_i) = \begin{cases} \lambda & \text{if } r_i \text{ is correct} \\ \min(0.5, \lambda) & \text{if } r_i \text{ is incorrect} \end{cases}, \text{where } \lambda = 1 - \frac{\text{len}(r_i) - \text{min\_len}}{\text{max\_len} - \text{min\_len}}. \quad (1)$$

4.2 Reflection Model for Online Sequential Revision

In §3, we employ a 32B LLM to perform revision to detect and remove overthinking tokens through a two-step annotation process. To achieve better scalability, we train a smaller reflection model, effectively providing sequential revision online, thus offering an additional dimension of scaling beyond traditional parallel sampling.

Reflection model training. To construct SFT data, we generate four responses per question on the training dataset using R1-7B, retaining only correct responses. Each chunk within these responses is labeled based on whether it contains the correct answer following §3.1. We then construct the SFT data using the question and all response chunks as input, training the reflection model to predict the category of each chunk. For efficiency, we adopt a concise output format, directly generating the ID and its category without additional deliberation, as detailed in Appendix A.

Online RL with sequential revision. As depicted in Figure 2, following the parallel sampling of multiple responses in online RL, we utilize the reflection model for sequential revision. Specifically, for the sampled response $S = \{s_1, ..., s_G\}$, we first use the reflection model $\text{M}_{\text{Reflect}}$ to remove overthinking tokens (red segment), copies the preceding segments from the original responses S (yellow segment), and prompt the policy model M_{Policy} to generate a final answer (blue segment), which forms the revision $S^r = \{s_1^r, ..., s_G^r\}$. Cases are shown in Appendix B.1. Ultimately, both the original responses S and the revised responses S^r are used as training data for the policy model, enabling an additional dimension of scaling and guiding the optimization direction of RL. Formally:

$$\{s_1, ..., s_G, s_1^r, ..., s_G^r\} \xrightarrow{\textbf{Online RL}} \text{M}_{\text{Policy}}, \text{where } s_i^r = \text{M}_{\text{Policy}}(\text{M}_{\text{Reflect}}(s_i)). \quad (2)$$

Equivalence to a partial reward. Given the substantial similarity between the pre- and post-revision responses (highlighted in blue), we can decompose the reward into two components. Formally, let a_i and a_i^r represent the advantages of the original response s_i and revised response s_i^r, respectively. The first component is the average reward across all tokens in both responses, formulated as $(a_i^r + a_i)/2$. The second component is a partial reward. Specifically, the overthinking tokens (red) receive a reward of $-(a_i^r - a_i)/2$, while the revised tokens (blue) receive $(a_i^r - a_i)/2$. When revision is successful, the length reward ensures $a_i^r > a_i$, effectively penalizing overthinking.

However, if both the original and revised responses are incorrect, it suggests a need for more extensive reasoning, making a preference for shorter outputs detrimental. Furthermore, if revision transforms a correct answer into an incorrect one, the opposite effect occurs, potentially encouraging overthinking. To mitigate these issues, we discard the revised responses in such cases and retain the original ones. Finally, following Yu et al. [45], when all generated trajectories yield incorrect answers, we skip these instances to avoid unintended optimization.

4.3 Reflection-Aware Reward Refinement

Reflection reward. To prevent the models from favoring short yet non-reflective responses due to length reward, we introduce a reflection reward based on the presence of reflective tokens, i.e., "wait", "alternatively", "check", and "but". We calculate the density of these tokens, and count closely clustered occurrences as a single instance to prevent consecutive reflective tokens within a single reflection. Formally, for the current response s_i, let N_{Token} be the total number of tokens in the response and N_{Reflect} be the number of reflective tokens. The reflection reward R_{Reflect} is:

$$R_{\text{Reflect}}(s_i) = \min(0, \frac{D_i}{D_{0.2}} - 1), \text{where } D_i = \frac{N_{\text{Reflect}}}{N_{\text{Token}}}. \quad (3)$$

$D_{0.2}$ represents the 0.2 quantile of the reflection density in the training data, and D_i is the density of r_i. This reward penalizes responses only when their density falls within the lowest 20% of observed densities, thereby preventing the reward from inadvertently promoting overthinking.

Length reward refinement. The length reward in Kimi K1.5 demonstrates a bias towards shorter responses even if the answers are incorrect. However, for challenging queries, more extensive reasoning is often required, conflicting with the current reward mechanism. Therefore, we follow Zhang and Zuo [46] to refine the length reward by grouping responses by query and setting the length reward to zero for all incorrect responses, which provides the definition of R_{RLen}.

5 Experiments

5.1 Experimental Setup

Common training configuration. Unless explicitly stated otherwise, we maintain the following experimental settings. We use the DeepScaleR 40k dataset [25] as the training data and DeepSeek-R1-Distill-Qwen-7B (R1-7B) as the base model. We only retain questions for which the model can provide at least one correct answer within 4 samples as the training data, resulting in 30k questions. We train for 1800 steps across all methods and baselines, with each batch containing 16 different questions and 4 different paths generated for each question, with a maximum generation length of 8k. We use a learning rate of 2×10^{-5}, integrating low-rank adaptation [14] with all attention and mlp parameters, setting the LoRA rank r to 16 and alpha to 32. During the data generation process, we follow R1 to use a temperature of 0.6 and a top_p of 0.95. Finally, the reflection model is fine-tuned from Qwen2.5-7B-Instruct for one epoch. During sequential revision, we set the temperature to 0 for the reflection model.

Online RL training. "$\text{M}_{\text{Reflect}}$" represents online revision in §4.2. "R_{Len}" represents adding the Kimi K1.5 length reward, "R_{RLen}" represents adding our refined length reward, and "$\text{R}_{\text{Reflect}}$" represents adding the reflection reward in §4.3. Accuracy reward is used in all GRPO settings. The primary baselines are 1) **GRPO**, which uses accuracy reward only. 2) **GRPO $\mathbf{R_{Len}}$**, which uses accuracy reward along with the Kimi K1.5 length reward.

Offline training. We also compare with commonly used offline baselines. We use the revised responses generated from the 32B model as the training data, following §3, with other generation settings consistent with the online approach. Only data that the answer is correct after revision is used for training. We then conduct training using 1) **SFT** [47]: We simply fine-tune the model using the aforementioned revised dataset. 2) **RPO** [30]: We treat the revised response as positive examples and the original paths as negative examples.

5.2 Main Result

Improved performance compared to offline method. The results are in Table 2. The final three rows showcase the performance of the reflection model, reward refinement, and their combined approach. While RPO shares similar ideas with the sequential revision, the primary distinction lies in the offline dataset. RPO achieves comparable performance at an 8k budget but significantly degrades on the three most challenging datasets at a 16k budget. In contrast, our method demonstrates better performance, highlighting the advantage of online training over offline training.

Improved efficiency and performance balance compared to online methods. GRPO, using only an accuracy reward, shows no significant performance improvement, suggesting that gains are not solely due to more training. Adding a length reward greatly reduces inference costs but severely hurts performance. Our method significantly improves the probability of solving problems within an 8k token budget (3.9% to 5.8% on average) and effectively shortens responses. With a sufficient 16k token budget, our approach reduces response length by 27% to 46% while maintaining accuracy.

Reflection model and reflection reward target distinct dimensions. The reflection model is more effective in reducing response length, whereas reflection reward contributes more to accuracy enhancement. By integrating both strategies, a balanced outcome could be achieved, yielding a 35%

	Method	GSM8K		Math500		Gaokao23		Amc23		Olympiad		Aime24		Average	
		Acc↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓
Budget: 8k	**Original**	91.58	100_{1312}	88.40	100_{3389}	77.40	100_{3270}	77.50	100_{4746}	51.70	100_{5467}	40.83	100_{6815}	71.24	100_{4166}
	SFT	89.92	64.63	89.20	69.25	77.92	72.91	82.81	72.67	52.44	82.06	41.25	89.98	72.26	75.25
	RPO	90.14	51.22	89.80	53.35	**83.12**	55.20	80.62	**59.54**	**58.52**	**61.24**	41.67	**74.78**	73.98	59.22
	GRPO	**92.80**	117.99	89.00	100.74	79.22	101.31	79.69	96.23	51.70	100.00	39.17	98.88	71.93	102.52
	GRPO R_{Len}	85.97	**32.01**	87.60	59.04	70.91	58.84	85.62	71.43	55.56	76.59	45.42	87.42	71.85	64.22
	GRPO R_{Len} $M_{Reflect}$	89.23	37.80	91.20	**48.66**	78.44	**52.23**	**86.56**	59.65	57.19	70.37	41.67	84.15	74.05	**58.81**
	GRPO $R_{RLen+Reflect}$	92.72	67.07	**91.40**	69.52	80.52	72.54	85.62	75.64	53.93	83.04	**47.92**	91.37	**75.35**	76.53
	GRPO $R_{RLen+Reflect}$ $M_{Reflect}$	89.99	53.12	89.60	61.79	81.30	61.19	85.62	71.39	55.11	77.81	42.92	88.45	74.09	73.86
Budget: 16k	**Original**	91.66	100_{1344}	92.00	100_{3893}	81.82	100_{3785}	88.12	100_{5840}	60.15	100_{7549}	48.33	100_{10460}	77.01	100_{5478}
	SFT	89.99	68.75	90.60	69.25	79.48	74.37	88.44	72.11	57.63	84.70	48.75	89.68	75.82	76.48
	RPO	90.14	50.00	89.80	46.90	**83.12**	47.77	82.19	**50.75**	58.37	**45.21**	42.92	**51.41**	74.42	**48.67**
	GRPO	**92.87**	116.52	**93.40**	99.20	82.34	100.26	87.19	97.31	60.44	96.78	50.42	97.18	77.78	101.21
	GRPO R_{Len}	85.97	**31.25**	88.40	56.43	72.21	55.88	87.81	65.65	59.56	69.20	50.00	76.96	73.99	59.23
	GRPO R_{Len} $M_{Reflect}$	89.23	36.90	92.40	**45.11**	79.74	**47.50**	88.12	56.04	**60.74**	63.52	47.92	75.82	76.36	54.15
	GRPO $R_{RLen+Reflect}$	92.72	65.48	92.80	66.71	81.82	68.27	88.75	72.17	58.81	77.37	**54.58**	86.21	**78.25**	72.70
	GRPO $R_{RLen+Reflect}$ $M_{Reflect}$	89.99	52.08	91.00	60.78	82.08	55.85	**89.38**	67.76	59.11	73.11	51.25	81.09	77.13	65.11

Table 2: Main results of our proposed methods. Most abbreviations align with Table 1. Baseline definitions are in §5.1. "GRPO R_{Len} $M_{Reflect}$" represents the addition of the reflection model, "GRPO $R_{RLen+Reflect}$" represents the addition of the reward optimization, and "GRPO $R_{RLen+Reflect}$ $M_{Reflect}$" represents the combination of both optimizations. "Budget" on the left is the max tokens allowed per question. Generation is forcibly stopped if this limit is exceeded.

	Method	GSM8K		Math500		Gaokao23		Amc23		Olympiad		Aime24		Average	
		Acc↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓
	Original	91.66	100_{1344}	92.00	100_{3893}	81.82	100_{3785}	88.12	100_{5840}	60.15	100_{7549}	48.33	100_{10460}	77.01	100_{5478}
	32B Revise	89.46	62.43	93.20	71.85	80.52	70.41	89.06	79.95	60.74	83.06	50.83	93.59	77.30	76.88
	+ Gold	88.78	54.09	92.00	57.80	79.48	59.71	87.19	67.71	60.15	73.48	51.67	92.31	76.54	67.52
Budget: 16k	**7B Revise Weak**	90.75	84.82	92.40	86.64	**82.86**	86.71	88.75	88.90	**62.22**	90.63	50.00	93.10	**77.83**	88.47
	Fixed Trunc (88.47)	89.84	84.23	92.00	86.75	81.04	86.92	88.12	89.30	61.19	91.27	47.50	93.55	76.61	88.67
	7B Revise Normal	90.22	79.02	91.80	81.09	**82.86**	81.82	89.06	83.80	61.78	87.55	49.17	91.75	77.48	84.17
	Fixed Trunc (84.17)	89.23	78.50	91.40	82.02	80.52	82.51	86.25	85.14	59.85	88.04	46.67	92.48	75.65	84.78
	7B Revise Strong	90.07	72.99	92.00	75.65	82.08	77.75	87.81	77.16	60.59	80.99	48.75	89.04	76.88	78.93
	Fixed Trunc (78.93)	88.32	72.77	90.00	76.73	79.22	78.76	80.94	80.02	58.96	82.08	45.83	90.12	73.88	80.08
	GRPO	**92.87**	116.52	**93.40**	99.20	82.34	100.26	87.19	97.31	60.44	96.78	50.42	97.18	77.78	101.21
	GRPO Gen8	92.42	103.35	92.00	97.82	82.34	100.24	**90.00**	92.74	58.07	98.68	50.00	93.60	77.47	97.74
	GRPO R_{Len}	85.97	31.25	88.40	56.43	72.21	55.88	87.81	65.65	59.56	69.20	50.00	76.96	73.99	59.23
	GRPO R_{Len} Gen8	85.52	**25.74**	88.20	51.68	72.21	56.78	83.12	63.78	57.78	71.53	**53.33**	82.51	73.36	58.67
	GRPO R_{Len} $M_{Reflect}$	89.23	36.90	92.40	**45.11**	79.74	**47.50**	88.12	**56.04**	60.74	**63.52**	47.92	**75.82**	76.36	**54.15**
	GRPO $R_{RLen+Reflect}$ $M_{Reflect}$	89.99	52.08	91.00	60.78	82.08	55.85	89.38	67.76	59.11	73.11	51.25	81.09	77.13	65.11

Table 3: Results of our proposed reflection model. "32B Revise" refers to the "Model Revise" results using a 32B LLM from Table 1. "7B Revise" uses our trained 7B revision model. "Fixed Trunc" indicates the use of a fixed truncation strategy with the same ratio as our corresponding method. Truncation strengths are defined in §5.3. The last three rows are copied from Table 2.

efficiency improvement without compromising performance. However, the improvement under an 8k budget remains limited. This can be attributed to the advantage of shorter generations under constrained budget, and the reflection model's inherent ability to foster reflection, as analyzed in §5.4. The effectiveness of R_{RLen} and $R_{Reflect}$ is further separately verified in Appendix B.2.

5.3 Reflection Model Analysis

Experimental setup. We follow the setup in §3.2 to verify our reflection model. We define three revision strengths: **Normal** truncates at the first identified correct answer; **Weak** truncates at the second such position; and **Strong** truncates before the first position where the truncation probability exceeds 0.25. If no such position exists, the Normal position is used. **Fixed Trunc** represents truncation at the closest sentence-ending position corresponding to the truncation ratio achieved by our method, and then forces the LRM to generate the final answer, as defined in §3.2.

Comparable performance to 32B at a significantly lower cost. Results are shown in Table 3. Our reflection model does not incorporate the gold answer as input. In comparison to the 32B model revised without gold input, our Strong strategy achieves a comparable compression ratio while

	Method	GSM8K		Math500		Gaokao23		Amc23		Olympiad		Aime24		Average	
		Acc↑	*Reflect*	*Acc*↑	*Reflect*	*Acc*↑	*Reflect*	*Acc*↑	*Reflect*	*Acc*↑	*Reflect*	*Acc*↑	*Reflect*	*Acc*↑	*Reflect*
	Original	91.66	105.48	92.00	107.75	81.82	90.94	88.12	102.72	60.15	80.09	48.33	84.82	77.01	95.30
Budget: 16k	**SFT**	89.99	87.73	90.60	104.43	79.48	79.97	88.44	90.00	57.63	77.31	48.75	82.30	75.82	86.96
	RPO	90.14	156.68	89.80	146.65	**83.12**	130.13	82.19	127.03	58.37	101.43	42.92	100.61	74.42	127.09
	GRPO	**92.87**	97.50	**93.40**	107.52	82.34	88.69	87.19	98.06	60.44	79.10	50.42	82.28	77.78	92.19
	GRPO R_{Len}	85.97	813.96	88.40	128.32	72.21	114.74	87.81	120.09	59.56	93.71	50.00	99.07	73.99	228.32
	GRPO R_{Len} $M_{Reflect}$	89.23	194.97	92.40	135.54	79.74	120.05	88.12	107.91	**60.74**	87.01	47.92	89.34	76.36	122.47
	GRPO $R_{RLen+Reflect}$	92.72	141.63	92.80	118.61	81.82	99.24	88.75	106.08	58.81	78.08	**54.58**	86.13	**78.25**	104.96
	GRPO $R_{RLen+Reflect}$ $M_{Reflect}$	89.99	150.87	91.00	126.51	82.08	109.91	**89.38**	105.89	59.11	89.82	51.25	90.25	77.13	112.21

Table 4: Reflection density of our proposed methods and baselines. Most abbreviations align with Table 2. "Reflect" represents the average number of tokens between each reflective token.

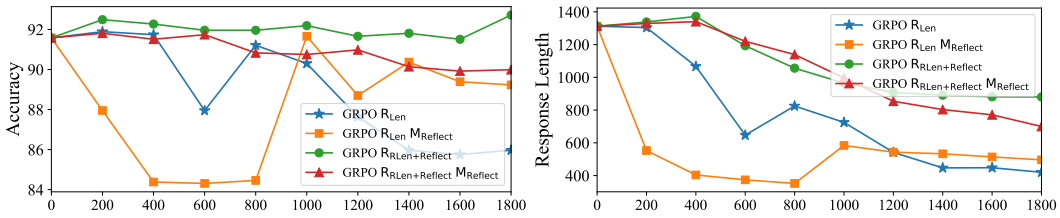

Figure 3: Accuracy and generation length changes during training on GSM8k. The x-axis represents the training steps, with a maximum of 1800 steps. The left plot shows the accuracy of question answering, and the right plot shows the average token consumption per answer. We present results for the GRPO baseline using only a length reward and our three proposed methods.

exhibiting minimal performance degradation, demonstrating that our model achieves comparable results at a substantially reduced cost. Furthermore, our method also outperforms fixed truncation, especially when the truncation ratio is large. Finally, we observe that under certain evaluations, the accuracy actually increases after revision. This indicates that the model may have already arrived at the correct answer but fails to explicitly state this conclusion under the 16k token budget.

REA-RL outperforms inference with reflection model. To ensure revision accuracy, we employ the Normal strategy during training. However, REA-RL with the reflection model achieves a higher compression ratio than the Strong strategy while maintaining comparable performance, despite the increased inference cost associated with the generate-then-revise approach of Strong strategy. This improvement can be attributed to the length reward and the online training, which iteratively train the model and provide further guidance for shortening already concise responses.

Online revision as an efficient scaling strategy. The reflection model provides revised responses, which doubles the dataset size. To evaluate whether other methods could yield similar benefits, we extend the GRPO baseline generation to eight paths, i.e., "Gen8", to verify whether parallel sampling scaling is more effective. However, Gen8 indicates no consistent improvement over parallel sampling of four paths, while our combination with sequential revision, under the same reward R_{Len}, demonstrates superior performance and truncation ratios, thereby establishing the enhanced efficacy of our scaling method compared to mere parallel sampling.

Regarding training time, we optimize the implementation by deferring the update of the vllm inference model, which enables parallel execution of training and data generation, resulting in approximately a twofold speedup. On 3 NVIDIA A800 80G GPUs, GRPO with sampling 4 paths requires 80 hours, while sampling 8 paths requires 110 hours. Our method, incorporating the reflection model, required 120 hours. Given the significant performance improvement, this additional computational cost is deemed acceptable. If training efficiency is a primary concern, utilizing the reflection reward alone also requires only 80 hours and achieves improved performance.

5.4 Reflection Ability Analysis for the Trained Model

Length reward impacts reflection frequency. We calculate the average number of tokens between reflective tokens in the responses, and the results are in Table 4. Introducing only a length reward in

online RL leads to a significant decrease in the frequency of reflective tokens in simple problems. However, for challenging problems, the model retains its reflection capability, as reflection is crucial for obtaining the accuracy reward. A case is in the middle of Figure 1, where the solution only performs planning in the "think" part and solves the problem without any reflection. Due to the minimal token consumption, an error occurs during planning. In contrast, our method preserves the style of R1, reasoning for sufficient tokens, and performing reflection after reasoning. To provide further analysis, we further illustrate the training dynamics of GSM8k in Figure 3, demonstrating that it sacrifices accuracy to achieve token length reduction, showing similar trends between accuracy and response length.

Both reflection model and reflection reward enhance reflection. Across the three easier datasets, our three approaches demonstrate an average increase of 49% in reflection probability and 7.5% performance improvement compared to GRPO R_{Len}, demonstrating that our methods effectively mitigate the issue of insufficient reflection. As illustrated in the training dynamics, the reflection reward facilitates a gradual improvement in efficiency with minimal change in accuracy. Besides, although using the reflection model itself initially leads to performance degradation similar to the baseline, the accuracy reward penalizes overly short responses that result in low accuracy, allowing the accuracy to recover. During the recovery, the reflection model can provide concise and more accurate training data, aiding the model in recovering performance without compromising efficiency. Finally, given that the reflection model inherently promotes reflection to some extent, its combination with the reflection reward can only achieve a trade-off rather than a substantial further improvement.

Mitigating overthinking on simple problems. LRM often exhibits overthinking on easier problems. Our method appropriately reduces reflection on easier problems while maintaining it on more difficult ones. While our approach increases the reflection frequency on simpler problems compared to R_{Len}, it remains lower than that of the original model. Specifically, on the three simplest datasets, we reduce the reflection frequency by 31%, and 45% efficiency enhancement. Whereas on challenging ones, the reduction of reflection is only 5%, and 27% efficiency enhancement. This demonstrates that our approach achieves a favorable balance, mitigating overthinking on simple problems while preserving reflection capabilities. Furthermore, we provide additional evidence demonstrating that our method reduces overthinking while preserving reflection capabilities in Appendix B.3.

6 Conclusion and Limitations

To enhance the inference efficiency of LRM without compromising model performance, we propose REflection-Aware online Reinforcement Learning, `REA-RL`, to achieve improved online scaling and better performance retention. Specifically, we introduce a reflection model for efficient scaling, offering sequential revision to augment parallel sampling data generation for online RL. Furthermore, we introduce a reflection reward to better maintain model performance. Experiments show that REA-RL reduces inference cost by 35% with no performance loss. Analysis proves that our method is effective by maintaining reflection frequency for challenging problems while appropriately reducing it for simpler ones, thus balancing performance and efficiency.

Our paper has the following limitations. First, our approach is only validated on distilled 7B LRMs. Due to the large model size and long training time, we do not perform validation on LRMs pre-trained from scratch, which aligns with prior work [2, 15, 32, 46]. Second, the detection method proposed in §3 is based on LLMs. While it outperforms fixed truncation, it cannot guarantee the complete elimination of overthinking. Nevertheless, our primary goal is to use it to train the reflection model, and as long as it can reduce overthinking to a certain extent, it can effectively shorten the reasoning process during iterative training. Finally, our scaling method introduces approximately 10% additional computational cost compared to using parallel scaling only. This is due to the use of sequential scaling, which results in poorer parallelism and longer runtime. However, considering our significant improvement and the cost-free improvement of reflection reward, this is acceptable.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023. URL https://arxiv.org/abs/2303.08774.

[2] Pranjal Aggarwal and Sean Welleck. L1: Controlling How Long A Reasoning Model Thinks With Reinforcement Learning. *arXiv preprint arXiv:2503.04697*, 2025. URL https://arxiv.org/abs/2503.04697.

[3] Daman Arora and Andrea Zanette. Training Language Models to Reason Efficiently. *arXiv preprint arXiv:2502.04463*, 2025. URL https://arxiv.org/abs/2502.04463.

[4] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. *arXiv preprint arXiv:2407.21787*, 2024. URL https://arxiv.org/abs/2407.21787.

[5] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs. *arXiv preprint arXiv:2412.21187*, 2024. URL https://arxiv.org/abs/2412.21187.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

[7] Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise Perplexity-Guided Refinement for Efficient Chain-of-Thought Reasoning in Large Language Models. *arXiv preprint arXiv:2502.13260*, 2025. URL https://arxiv.org/abs/2502.13260.

[8] Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, et al. Kimi k1.5: Scaling Reinforcement Learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025. URL https://arxiv.org/abs/2501.12599.

[9] Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. *arXiv preprint arXiv:2501.04519*, 2025. URL https://arxiv.org/abs/2501.04519.

[10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

[11] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-Budget-Aware LLM Reasoning. *arXiv preprint arXiv:2412.18547*, 2024. URL https://arxiv.org/abs/2412.18547.

[12] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.211. URL https://aclanthology.org/2024.acl-long.211/.

[13] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving

with the MATH dataset. In Joaquin Vanschoren and Sai-Kit Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html.

[14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[15] Chengsong Huang, Langlin Huang, Jixuan Leng, Jiacheng Liu, and Jiaxin Huang. Efficient Test-Time Scaling via Self-Calibration. *arXiv preprint arXiv:2503.00031*, 2025. URL https://arxiv.org/abs/2503.00031.

[16] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. OpenAI o1 System Card. *arXiv preprint arXiv:2412.16720*, 2024. URL https://arxiv.org/abs/2412.16720.

[17] Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3oT: Generating Shorter Chain-of-Thought without Compromising Effectiveness. *arXiv preprint arXiv:2412.11664*, 2024. URL https://arxiv.org/abs/2412.11664.

[18] Logan Kilpatrick. Gemini 2.5 Pro Preview: Even better coding performance. Google for Developers, 2025.

[19] Minpeng Liao, Chengxi Li, Wei Luo, Wu Jing, and Kai Fan. MARIO: MAth reasoning with code interpreter output - a reproducible pipeline. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 905–924, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.53. URL https://aclanthology.org/2024.findings-acl.53/.

[20] Zicheng Lin, Tian Liang, Jiahao Xu, Qiuzhi Lin, Xing Wang, Ruilin Luo, Chufan Shi, Siheng Li, Yujiu Yang, and Zhaopeng Tu. Critical Tokens Matter: Token-Level Contrastive Estimation Enhances LLM's Reasoning Capability. *arXiv preprint arXiv:2411.19943*, 2024. URL https://arxiv.org/abs/2411.19943.

[21] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

[22] Jie Liu, Zhanhui Zhou, Jiaheng Liu, Xingyuan Bu, Chao Yang, Han-Sen Zhong, and Wanli Ouyang. Iterative Length-Regularized Direct Preference Optimization: A Case Study on Improving 7B Language Models to GPT-4 Level. *arXiv preprint arXiv:2406.11817*, 2024. URL https://arxiv.org/abs/2406.11817.

[23] Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can Language Models Learn to Skip Steps? *arXiv preprint arXiv:2411.01855*, 2024. URL https://arxiv.org/abs/2411.01855.

[24] Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-Pruner: Length-Harmonizing Fine-Tuning for O1-Like Reasoning Pruning. *arXiv preprint arXiv:2501.12570*, 2025. URL https://arxiv.org/abs/2501.12570.

[25] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, et al. DeepScaleR: Surpassing O1-preview with a 1.5B model by scaling RL, 2025.

[26] Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. LLM Critics Help Catch LLM Bugs. *arXiv preprint arXiv:2407.00215*, 2024. URL https://arxiv.org/abs/2407.00215.

[27] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. S1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025. URL https://arxiv.org/abs/2501.19393.

[28] Tergel Munkhbat, Namgyu Ho, Seo Hyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-Training Elicits Concise Reasoning in Large Language Models. *arXiv preprint arXiv:2502.20122*, 2025. URL https://arxiv.org/abs/2502.20122.

[29] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.57. URL https://aclanthology.org/2024.findings-acl.57/.

[30] Richard Yuanzhe Pang, Weizhe Yuan, Kyunghyun Cho, He He, Sainbayar Sukhbaatar, and Jason Weston. Iterative Reasoning Preference Optimization. *arXiv preprint arXiv:2404.19733*, 2024. URL https://arxiv.org/abs/2404.19733.

[31] Xiaoye Qu, Yafu Li, Zhaochen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, et al. A Survey of Efficient Reasoning for Large Reasoning Models: Language, Multimodality, and Beyond. *arXiv preprint arXiv:2503.21614*, 2025. URL https://arxiv.org/abs/2503.21614.

[32] Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing Test-Time Compute via Meta Reinforcement Fine-Tuning. *arXiv preprint arXiv:2503.07572*, 2025. URL https://arxiv.org/abs/2503.07572.

[33] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024. URL https://arxiv.org/abs/2412.15115.

[34] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[35] Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo Lian. DAST: Difficulty-Adaptive Slow-Thinking for Large Reasoning Models. *arXiv preprint arXiv:2503.04472*, 2025. URL https://arxiv.org/abs/2503.04472.

[36] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL https://arxiv.org/abs/2408.03314.

[37] Yunhao Tang, Daniel Zhaohan Guo, Zeyu Zheng, Daniele Calandriello, Yuan Cao, Eugene Tarassov, Rémi Munos, Bernardo Ávila Pires, Michal Valko, Yong Cheng, et al. Understanding the performance gap between online and offline alignment algorithms. *arXiv preprint arXiv:2405.08448*, 2024. URL https://arxiv.org/abs/2405.08448.

[38] Qwen Team. QwQ-32B: Embracing the power of reinforcement learning, 2025.

[39] Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. TokenSkip: Controllable Chain-of-Thought Compression in LLMs. *arXiv preprint arXiv:2502.12067*, 2025. URL https://arxiv.org/abs/2502.12067.

[40] Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P. Lillicrap, Kenji Kawaguchi, and Michael Shieh. Monte Carlo Tree Search Boosts Reasoning via Iterative Preference Learning. *arXiv preprint arXiv:2405.00451*, 2024. URL https://arxiv.org/abs/2405.00451.

[41] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of Draft: Thinking Faster by Writing Less. *arXiv preprint arXiv:2502.18600*, 2025. URL https://arxiv.org/abs/2502.18600.

[42] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. *arXiv preprint arXiv:2409.12122*, 2024. URL https://arxiv.org/abs/2409.12122.

[43] Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying Long Chain-of-Thought Reasoning in LLMs. *arXiv preprint arXiv:2502.03373*, 2025. URL https://arxiv.org/abs/2502.03373.

[44] Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling System 2 into System 1. *arXiv preprint arXiv:2407.06023*, 2024. URL https://arxiv.org/abs/2407.06023.

[45] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025. URL https://arxiv.org/abs/2503.14476.

[46] Jixiao Zhang and Chunsheng Zuo. GRPO-LEAD: A Difficulty-Aware Reinforcement Learning Approach for Concise Mathematical Reasoning in Language Models. *arXiv preprint arXiv:2504.09696*, 2025. URL https://arxiv.org/abs/2504.09696.

[47] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. Instruction Tuning for Large Language Models: A Survey. *arXiv preprint arXiv:2308.10792*, 2023. URL https://arxiv.org/abs/2308.10792.

A Prompts

We introduce all prompts used in the main text. Parts enclosed in "{}" represent external input.

- **The Prompts for Detecting Overthinking:** These prompts are used in §3.1. We first segment the response into chunks, specifically by splitting it at every two newline characters. To prevent a single formula from being split across multiple chunks, we merge chunks until the combined chunk ends with a sentence-ending punctuation mark or exceeds 128 tokens. To prevent the model from generating excessively long labeling strings, we input a maximum total length of 1k tokens at a time. Subsequently, we first feed all the chunks into the first prompt for an initial detection. Each chunk subsequently labeled as "Right Result" is then fed into the second prompt for secondary labeling. "With / without Gold Answer" indicates whether the gold answer is included as input.
- **The SFT Prompt for Reflection Model Training:** These prompts constitute the SFT data we constructed in §4.2 to train a 7B reflection model. We employ simpler definitions and non-chain-of-thought output to ensure both effectiveness and efficiency. The gold answer is not used as input to ensure usability during training and evaluation.

The First Prompt for Detecting Overthinking with Gold Answer

Question: {Question}
Gold Answer: {Answer}
Response: {All Chunked Responses}
You are provided with a math Question, a Gold Answer and a model-generated Response. The response is divided into {N} parts. For each part, analyze it and classify it based on its relationship to the provided context. For each part, assign one of the following labels:
- Reasoning: The part represents the reasoning process that leads to the answer.
- Right Result: The part is the answer provided by the model, where the model may provide the answer in the middle of its response, and the answer aligns with the Gold Answer.
- Wrong Result: Same as Right Result, but the answer does not align with the Gold Answer.

For each of the {N} parts, please reply in format:
[1]. Think: [Explanation for label choice]
Label: Reasoning/Right Result/Wrong Result
[2]. Think: [Explanation for label choice]
Label: Reasoning/Right Result/Wrong Result
...

The First Prompt for Detecting Overthinking without Gold Answer

Question: {Question}
Response: {All Chunked Responses}
You are provided with a math Question and a model-generated Response. The response is divided into {N} parts. For each part, analyze it and classify it based on its relationship to the provided context. For each part, assign one of the following labels:
- Reasoning: The part represents the reasoning process that leads to the answer.
- Right Result: The part is the answer provided by the model, where the model may provide the answer in the middle of its response, and the answer align with the Gold Answer.
- Wrong Result: Same as Right Result, but the answer do not align with the Gold Answer.

For each of the {N} parts, please reply in format:
[1]. Think: [Explanation for label choice]
Label: Reasoning/Right Result/Wrong Result
[2]. Think: [Explanation for label choice]
Label: Reasoning/Right Result/Wrong Result
...

- **The Prompt for Math Evaluation:** We follow Yang et al. [42] to design the prompts for math evaluation. When forcing the model to complete generation and provide the final answer, we append "</think> **Final Answer:**" after the thought process and allow the model to continue generating up to 1k tokens. During training, to ensure efficiency, we reduce the budget to 256 tokens.

The Second Prompt for Detecting Overthinking with Gold Answer

Question: {Question}
Gold Answer: {Answer}
Response: {One Chunked Response}
Evaluate whether the model correctly answered the question. As long as the model provides the correct result, it counts as correct, regardless of format or wording. The response I provided is part of the complete response, so thereś no need to include the entire reasoning process. Please judge only if the model has provided the correct answer up to this point. Please reason step by step first after "Reasoning:", then answer only with Yes or No after "Answer:".

The Second Prompt for Detecting Overthinking without Gold Answer

Question: {Question}
Response: {One Chunked Response}
Evaluate whether the model has already answered the question. The response I provided is part of the complete response, so thereś no need to include the entire reasoning process. Please judge only if the model has provided the answer up to this point. Please reason step by step first after "Reasoning:", then answer only with Yes or No after "Answer:".

The SFT Prompt for Reflection Model Training

Question: {Question}
Response: {One Chunked Response}
You are provided with a math Question and a model-generated Response. The response is divided into {N} parts. For each part, analyze it and classify it based on its relationship to the provided context. For each part, assign one of the following labels:
- Think: The part represents the reasoning process that leads to the answer.
- Result: The part is the answer provided by the model, where the model may provide the answer in the middle of its response.

For each of the {N} parts, please reply in format:
[1]. Think/Result
[2]. Think/Result
...

The Prompt for Math Evaluation

Please reason step by step, and put your final answer within \boxed{}.
Question: {Question}

B Further Analysis

B.1 Case Study for REA-RL

To illustrate how our method works, we provide a case study demonstrating the workflow of online data generation for online RL, containing both parallel sampling and sequential revision, as shown in Figure 4. Specifically, we first perform parallel sampling, obtaining the yellow and red parts. Here, the red part represents the overthinking section, which is detected by the reflection model. Subsequently, we perform sequential revision by removing the red part and allowing the policy model to finish the response with the blue part. In detail, we force the termination of the thinking process by adding a "</think>" token and enforce answer generation by adding "**Final Answer:**" to avoid further redundant reasoning. Based on this process, the blue part is generally much shorter than the red part, thus providing cases with less overthinking online.

Since we enforce a limit that no more than half of the tokens in the original response can be removed, and the reflection model cannot always identify the first correct answer, not all additional reflections can be removed. Therefore, the yellow part may also contain some reflection, which also helps REA-RL retain its reflection ability. Additionally, in the second case, the model fails to complete generation within the 8k token budget, resulting in the answer not being formatted as required (within \boxed), and thus marked as incorrect. However, the model has already generated the correct answer, making its response correct after revision. This explains why truncation can sometimes improve performance.

Question: Determine the minimum possible value of the sum $\frac{a}{3b} + \frac{b}{6c} + \frac{c}{9a}$, where a, b, and c are positive real numbers.
Answer: $\frac{1}{\sqrt[3]{6}}$

s_1

<think> Okay, so I have this problem where I need to find the minimum value of the expression ... ... Hence, the minimum value of S is: $S \geq 3 \times \frac{1}{3\times\sqrt[3]{6}} = \frac{1}{\sqrt[3]{6}}$ ... ... Wait, but let me double-check that calculation because sometimes when I manipulate cube roots, I might make a mistake. ... ... so $S = 3 \times \frac{1}{3\sqrt[3]{6}} = \frac{1}{\sqrt[3]{6}}$, which confirms that the minimum value is indeed $\frac{1}{\sqrt[3]{6}}$. 3638 tokens Advantage: 0.95

s_1^r

But let me just confirm if this makes sense. ... ... So, after careful consideration and verification, the minimum value of the given expression is $\frac{1}{\sqrt[3]{6}}$. **Final Answer** The minimum possible value is \boxed{$\frac{1}{\sqrt[3]{6}}$}. </think> To determine ... ... Therefore, the minimum possible value of the given expression is \boxed{$\frac{1}{\sqrt[3]{6}}$}. 2307 tokens Advantage: 0.95-0.45

</think> **Final Answer:** The minimum possible value of the sum is \boxed{$\frac{1}{\sqrt[3]{6}}$}. 35 tokens Advantage: 0.95+0.45

s_2

<think> Okay, so I have this problem where I need to find the minimum value of the expression $\frac{a}{3b} + \frac{b}{6c} + \frac{c}{9a}$, where a, b, and c are positive real numbers. ... ... So, the minimal value is $\frac{1}{3\sqrt[3]{6}}$. Alternatively, it can be expressed as $\frac{\sqrt[3]{36}}{18}$, but both are correct. Wait, let me compute $\sqrt[3]{36}$ ... ... which is $\frac{3}{\sqrt[3]{162}} = \frac{3}{3\times\sqrt[3]{6}} = \frac{1}{\sqrt[3]{6}}$. 7494 tokens Advantage: -1.10

s_2^r

Wait, that's different. Wait, hold on, perhaps I made a miscalculation earlier. ... ... Wait, no, wait: $\frac{3}{\sqrt[3]{162}} = \frac{3}{3\times\sqrt[3]{6}} = \frac{1}{\sqrt[3]{6}} \approx 0.55$. So, yes, that's correct. But when I did the substitution, I had an expression that 506 tokens Advantage: -1.10-0.83

</think> **Final Answer:** The minimum possible value is \boxed{$\frac{1}{\sqrt[3]{6}}$}. 30 tokens Advantage: -1.10+0.83

Figure 4: Case Study for REA-RL of online data generation. We illustrate how the parallel sampling and sequential revision parts work. Specifically, the yellow, red, and blue parts in this figure correspond to the tokens of the same colors in Figure 2. Since the yellow parts in both completion and revision are identical, they are shown only once in this figure.

B.2 Ablation Study of Reward Refinement

To further validate our proposed reward refinement scheme, including the improvements to reflection reward and length reward, the results are presented in Table 5.

Refined length accelerates response shortening. We evaluate the performance of the length reward before and after optimization (Len vs. RLen) when using only the accuracy reward and when using both the accuracy reward and the reflection reward. The results demonstrate that RLen can significantly reduce the number of tokens used while maintaining or even improving performance. We attribute this to the removal of positive signals for incorrect answers, which avoids encouraging erroneous responses and reduces input noise, thereby accelerating convergence.

Reflection reward improves performance but reduces efficiency. Across all experiments, we observe an average performance gain of 5% compared to the scenarios without the reflection reward. However, this improvement comes at the cost of an average 16% reduction in the truncation ratio. Nevertheless, we believe that maintaining performance is a more critical objective than shortening the response. Only with the addition of the reflection reward can we achieve a reduction in model response length without sacrificing model performance, and we anticipate further shortening of the response with continued training, as illustrated in Figure 3. Conversely, continuing training with only the length reward may lead to a further decline in accuracy.

Quantile hyperparameter settings in reflection reward. In the reflection reward, we utilize the 0.2 quantile of the reflection density during training, i.e., $D_{0.2}$. To investigate the impact of other quantiles on the experimental results, we further explore the effects of the 0.1 and 0.4 quantiles. Specifically, $D_{0.1} = 1/299$, $D_{0.2} = 1/225$, and $D_{0.4} = 1/157$, where the denominator represents the number of tokens between reflection tokens at that quantile. The results indicate that as the quantile increases, the penalty for reflection density becomes more severe, leading to longer responses. However, its impact on performance is relatively small, especially with a sufficient budget. This demonstrates that the effectiveness of the reflection reward stems from discouraging non-reflection behavior rather than unconditionally encouraging reflection.

	Method	GSM8K		Math500		Gaokao23		Amc23		Olympiad		Aime24		Average	
		Acc↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓
Budget: 8k	**Original**	91.58	100_{1312}	88.40	100_{3389}	77.40	100_{3270}	77.50	100_{4746}	51.70	100_{5467}	40.83	100_{6815}	71.24	100_{4166}
	GRPO R_{Len}	85.97	32.01	87.60	59.04	70.91	58.84	85.62	71.43	**55.56**	76.59	45.42	87.42	71.85	64.22
	GRPO R_{RLen}	86.05	**29.42**	88.20	**50.19**	68.83	**51.10**	**86.88**	**63.27**	53.93	**71.67**	**48.75**	**84.92**	72.11	**58.43**
	GRPO $R_{Len+Reflect}$	92.95	89.63	**91.60**	81.74	**81.30**	84.68	83.12	87.00	55.11	88.38	44.58	95.52	74.78	87.83
	w/ $D_{0.1}$	92.34	80.26	**91.60**	76.54	79.22	78.87	84.06	82.64	53.78	87.91	40.00	93.25	73.50	83.25
	w/ $D_{0.4}$	**93.25**	108.46	88.00	93.24	80.52	96.51	81.88	92.82	52.59	95.43	37.50	100.18	72.29	97.77
	GRPO $R_{RLen+Reflect}$	92.72	67.07	91.40	69.52	80.52	72.54	85.62	75.64	53.93	83.04	47.92	91.37	**75.35**	76.53
Budget: 16k	**Original**	91.66	100_{1344}	92.00	100_{3893}	81.82	100_{3785}	88.12	100_{5840}	60.15	100_{7549}	48.33	100_{10460}	77.01	100_{5478}
	GRPO R_{Len}	85.97	31.25	88.40	56.43	72.21	55.88	87.81	65.65	59.56	69.20	50.00	76.96	73.99	59.23
	GRPO R_{RLen}	86.05	**28.72**	89.20	**46.62**	69.35	**47.03**	88.12	**56.59**	56.74	**64.47**	54.17	**75.40**	73.94	**53.14**
	GRPO $R_{Len+Reflect}$	92.95	88.10	**93.20**	79.58	82.86	79.39	87.50	83.30	59.70	83.32	51.67	89.54	77.98	83.87
	w/ $D_{0.1}$	92.34	78.65	**93.20**	73.67	81.56	77.31	89.06	80.62	**61.33**	82.50	47.92	91.85	77.57	80.77
	w/ $D_{0.4}$	**93.40**	107.59	91.60	93.68	**83.90**	92.26	**89.69**	92.47	60.15	93.16	46.25	101.19	77.50	96.72
	GRPO $R_{RLen+Reflect}$	92.72	65.48	92.80	66.71	81.82	68.27	88.75	72.17	58.81	77.37	**54.58**	86.21	**78.25**	72.70

Table 5: Results of the ablation study. The table presents two sets of experiments using R_{Len} and R_{RLen} to demonstrate the effectiveness of our length reward optimization, as well as an ablation study on the hyperparameters of the reflection reward. "w/ $D_{0.1}$" and "w/ $D_{0.4}$" are defined in Equation 3, representing the use of the 0.1 and 0.4 quantiles of the reflection density for the reflection reward, respectively. Other abbreviations are defined in Table 2.

B.3 Overthinking Analysis After Training

Significantly lower overthinking. To verify that our method alleviates the overthinking issue, we employ the same evaluation approach as in §3 on our trained model, which uses an LLM to identify overthinking tokens and remove them with model reflection. The results are presented in Table 6. Compared to directly truncating the response from the original model, our approach exhibits a considerably lower truncation ratio for overthinking tokens, especially for methods employing our reflection model for training. This demonstrates that REA-RL effectively mitigates the issue of overthinking.

Preserving reflection capability. While our method reduces the truncation ratio, often indicating less additional reflection, it does not eliminate it entirely. A certain proportion of truncation is still maintained across our methods, particularly in experiments utilizing our reflection reward. This indicates that our method still retains a certain token budget for the final reflection, thus proving that REA-RL preserves the reflection capability.

	Method	GSM8K		Math500		Gaokao23		Amc23		Olympiad		Aime24		Average	
		Acc↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓	*Acc*↑	*TR*↓
Budget: 16k	**Original**	91.66	100_{1344}	92.00	100_{3893}	81.82	100_{3785}	88.12	100_{5840}	60.15	100_{7549}	48.33	100_{10460}	77.01	100_{5478}
	Model Reflect	89.46	62.43	**93.20**	71.85	80.52	70.41	89.06	79.95	**60.74**	83.06	50.83	93.59	77.30	76.88
	+ Gold	88.78	**54.09**	92.00	**57.80**	79.48	**59.71**	87.19	**67.71**	60.15	**73.48**	51.67	**92.31**	76.54	**67.52**
	GRPO $R_{RLen+Reflect}$	**92.72**	100_{880}	92.80	100_{2597}	81.82	100_{2584}	88.75	100_{4215}	58.81	100_{5841}	**54.58**	100_{9018}	**78.25**	100_{4189}
	Model Reflect	92.42	72.73	93.00	78.17	80.26	75.46	87.19	84.01	57.93	86.77	**54.58**	94.72	77.56	81.98
	+ Gold	91.74	68.75	91.60	69.77	79.74	70.51	85.94	77.27	57.19	79.27	**54.58**	93.70	76.80	76.54
	GRPO R_{Len} $M_{Reflect}$	89.23	100_{496}	92.40	100_{1756}	79.74	100_{1798}	88.12	100_{3273}	**60.74**	100_{4795}	47.92	100_{7931}	76.36	100_{3342}
	Model Reflect	88.02	90.73	91.40	92.54	79.22	92.99	88.75	93.92	59.41	91.60	45.42	95.51	75.37	92.88
	+ Gold	87.72	88.91	89.80	84.85	78.96	88.65	86.56	90.53	58.96	83.96	46.25	95.21	74.71	88.68
	GRPO $R_{RLen+Reflect}$ $M_{Reflect}$	89.99	100_{700}	91.00	100_{2366}	**82.08**	100_{2114}	**89.38**	100_{3957}	59.11	100_{5519}	51.25	100_{8482}	77.13	100_{3856}
	Model Reflect	90.30	86.29	91.60	88.08	80.78	88.69	88.75	90.90	60.59	90.20	50.42	96.24	77.07	90.07
	+ Gold	89.46	80.86	90.40	80.85	79.74	82.64	87.19	88.10	59.41	82.68	50.83	95.48	76.17	85.10

Table 6: Results of the overthinking analysis. Each group of results is divided into three rows. The first row shows the performance of the original model or the trained model directly. The following two rows represent the results of shortening the responses of the first-row model using different methods to remove overthinking. "Model Reflect" and "+ Gold" are defined in Table 3, representing revision using the 32B model. Other abbreviations are defined in Table 2.